NextAuth.JS Implementation

Goal

Build a **Next.js 16** app with:

- **Login page**
- **Google login**
- **GitHub login**
- **Protected `/profile` page**
- If user opens `/profile` without login → **redirect to `/login`**
- If already logged in and opens `/login` → **redirect to `/profile`**

This guide is specifically for:

- **Next.js 16**
- **JavaScript**
- **App Router**
- `src/app` **folder**
- **NextAuth/Auth.js v5 style**
- `proxy.js` (not middleware)

---

1) Create a fresh Next.js app

If not already created:

```
npx create-next-app@latest my-auth-app
```

While creating, choose:

- **JavaScript**
- **App Router**

- **Use `src/` directory → Yes**

Then go inside project:

```
cd my-auth-app
```

---

2) Install correct NextAuth version

⚠️ Very important: for this setup, install **beta** version.

```
npm install next-auth@beta
```

---

3) Generate `AUTH_SECRET`

Run this in terminal:

```
openssl rand -base64 32
```

Copy the output.

You will use it in `.env.local`.

---

4) Create `.env.local`

Create this file in your **project root**

(same place where `package.json` exists):

```
.env.local
```

Add this:

```
AUTH_SECRET=PASTE_YOUR_GENERATED_SECRET_HERE
NEXTAUTH_URL=http://localhost:3000
```

```
AUTH_GOOGLE_ID=
AUTH_GOOGLE_SECRET=

AUTH_GITHUB_ID=
AUTH_GITHUB_SECRET=
```

⚠️ Don't fill provider IDs yet. First create OAuth apps.

---

5) Create Google OAuth App

Open:

👉 **Google Cloud Console**

Go to:

```
APIs & Services → Credentials
```

Step-by-step

5.1 Create a Project

- Click **Select Project**
- Click **New Project**
- Give any name, e.g.

```
NextAuth Demo
```

5.2 Configure OAuth Consent Screen

Go to:

```
APIs & Services → OAuth consent screen
```

Fill:

- **App name** → `NextAuth Demo`
- **User support email** → your email
- **Developer contact email** → your email

Click **Save and Continue** until done.

5.3 Create OAuth Client

Go to:

```
APIs & Services → Credentials
```

Click:

```
Create Credentials → OAuth client ID
```

Choose:

```
Application type: Web application
```

Give name:

```
NextAuth Google Login
```

Add:

Authorized JavaScript Origins

```
http://localhost:3000
```

Authorized Redirect URIs

```
http://localhost:3000/api/auth/callback/google
```

Click **Create**

5.4 Copy Google credentials

You will get:

- **Client ID**
- **Client Secret**

Paste them in `.env.local`:

```
AUTH_GOOGLE_ID=your_google_client_id
AUTH_GOOGLE_SECRET=your_google_client_secret
```

---

6) Create GitHub OAuth App

Open:

👉 **GitHub**

Go to:

```
Settings → Developer settings → OAuth Apps
```

Step-by-step

Click:

```
New OAuth App
```

Fill:

Application name

```
NextAuth Demo
```

Homepage URL

```
http://localhost:3000
```

Authorization callback URL

```
http://localhost:3000/api/auth/callback/github
```

Click:

```
Register application
```

Now you will get:

- **Client ID**

Then click:

```
Generate a new client secret
```

Copy:

- **Client ID**
- **Client Secret**

Paste in `.env.local`:

```
AUTH_GITHUB_ID=your_github_client_id
AUTH_GITHUB_SECRET=your_github_client_secret
```

---

7) Final `.env.local` should look like this

```
AUTH_SECRET=your_generated_secret
NEXTAUTH_URL=http://localhost:3000
```

```
AUTH_GOOGLE_ID=your_google_client_id
AUTH_GOOGLE_SECRET=your_google_client_secret

AUTH_GITHUB_ID=your_github_client_id
AUTH_GITHUB_SECRET=your_github_client_secret
```

---

8) Create `src/auth.js`

Create this file:

```
src/auth.js
```

Paste this:

```
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";
import GitHub from "next-auth/providers/github";

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    Google({
      clientId: process.env.AUTH_GOOGLE_ID,
      clientSecret: process.env.AUTH_GOOGLE_SECRET,
    }),
    GitHub({
      clientId: process.env.AUTH_GITHUB_ID,
      clientSecret: process.env.AUTH_GITHUB_SECRET,
    }),
  ],
  session: {
    strategy: "jwt",
  },
  pages: {
    signIn: "/login",
```

```
  },
});
```

---

9) Create NextAuth API route

Create this file:

```
src/app/api/auth/[...nextauth]/route.js
```

Paste this:

```
import { handlers } from "@/auth";

export const { GET, POST } = handlers;
```

---

10) Create `proxy.js` (Next.js 16)

⚠️ In **Next.js 16**, use `proxy.js`, not `middleware.js`.

Create this file:

```
src/proxy.js
```

Paste this:

```
import { NextResponse } from "next/server";
import { auth } from "@/auth";

export const proxy = auth((req) => {
  const isLoggedIn = !!req.auth;
  const { pathname } = req.nextUrl;

  const isProfilePage = pathname.startsWith("/profile");
  const isLoginPage = pathname.startsWith("/login");
```

```
  if (isProfilePage && !isLoggedIn) {
    return NextResponse.redirect(new URL("/login", req.url));
  }

  if (isLoginPage && isLoggedIn) {
    return NextResponse.redirect(new URL("/profile", req.ur
l));
  }

  return NextResponse.next();
});

export const config = {
  matcher: ["/profile/:path*", "/login"],
};
```

---

11) Create Session Provider

Create this file:

```
src/app/providers.js
```

Paste:

```
"use client";

import { SessionProvider } from "next-auth/react";

export default function Providers({ children }) {
  return <SessionProvider>{children}</SessionProvider>;
}
```

---

12) Update `src/app/layout.js`

Open:

```
src/app/layout.js
```

Replace with:

```
import Providers from "./providers";

export const metadata = {
  title: "NextAuth Demo",
  description: "Google and GitHub Login Demo",
};

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}
```

---

13) Create Login Page

Create this file:

```
src/app/login/page.js
```

Paste this:

```
"use client";
```

```
import { signIn } from "next-auth/react";

export default function LoginPage() {
  return (
    <div style={styles.container}>
      <div style={styles.card}>
        <h1>Login</h1>
        <p>Sign in to continue</p>

        <button
          onClick={() => signIn("google", { callbackUrl: "/pr
ofile" })}
          style={styles.button}
        >
          Continue with Google
        </button>

        <button
          onClick={() => signIn("github", { callbackUrl: "/pr
ofile" })}
          style={styles.button}
        >
          Continue with GitHub
        </button>
      </div>
    </div>
  );
}

const styles = {
  container: {
    height: "100vh",
    display: "flex",
    justifyContent: "center",
    alignItems: "center",
    background: "#f4f4f4",
```

```
  },
  card: {
    background: "white",
    padding: "30px",
    borderRadius: "12px",
    width: "350px",
    textAlign: "center",
    boxShadow: "0 4px 15px rgba(0,0,0,0.1)",
  },
  button: {
    display: "block",
    width: "100%",
    marginTop: "15px",
    padding: "12px",
    borderRadius: "8px",
    border: "none",
    background: "#111",
    color: "white",
    cursor: "pointer",
    fontSize: "16px",
  },
};
```

---

14) Create Profile Page

Create this file:

```
src/app/profile/page.js
```

Paste this:

```
import { auth, signOut } from "@/auth";
import { redirect } from "next/navigation";
```

```
export default async function ProfilePage() {
  const session = await auth();

  if (!session) {
    redirect("/login");
  }

  return (
    <div style={styles.container}>
      <div style={styles.card}>
        <h1>Profile</h1>

        <img
          src={session.user?.image}
          alt="Profile"
          width="100"
          height="100"
          style={{
            borderRadius: "50%",
            objectFit: "cover",
            marginBottom: "20px",
          }}
        />

        <p><strong>Name:</strong> {session.user?.name}</p>
        <p><strong>Email:</strong> {session.user?.email}</p>

        <form
          action={async () => {
            "use server";
            await signOut({ redirectTo: "/login" });
          }}
        >
          <button type="submit" style={styles.button}>
            Logout
          </button>
```

```
        </form>
      </div>
    </div>
  );
}

const styles = {
  container: {
    height: "100vh",
    display: "flex",
    justifyContent: "center",
    alignItems: "center",
    background: "#f4f4f4",
  },
  card: {
    background: "white",
    padding: "30px",
    borderRadius: "12px",
    width: "400px",
    textAlign: "center",
    boxShadow: "0 4px 15px rgba(0,0,0,0.1)",
  },
  button: {
    marginTop: "20px",
    padding: "10px 20px",
    borderRadius: "8px",
    border: "none",
    background: "crimson",
    color: "white",
    cursor: "pointer",
    fontSize: "16px",
  },
};
```

15) Redirect `/` to `/login`

Create this file:

```
src/app/page.js
```

Paste this:

```
import { redirect } from "next/navigation";

export default function HomePage() {
  redirect("/login");
}
```

---

16) Final folder structure

Your project should look like this:

```
src
├── app
│   ├── api
│   │   └── auth
│   │       └── [...nextauth]
│   │           └── route.js
│   ├── login
│   │   └── page.js
│   ├── profile
│   │   └── page.js
│   ├── layout.js
│   ├── page.js
│   └── providers.js
├── auth.js
└── proxy.js
```

And root:

```
my-auth-app/
├── .env.local
├── package.json
├── src/
```

---

17) Restart properly

⚠️ After adding env values or auth files, restart properly:

```
rm -rf .next
npm run dev
```

---

18) Test important auth route first

Open this in browser:

```
http://localhost:3000/api/auth/session
```

Expected:

You should see JSON like:

```
{}
```

or session data if logged in.

If you see HTML error page, your auth setup is broken.

---

19) Run the app

Open:

```
http://localhost:3000
```

---

20) Test the flow

Case 1: Open app

Go to:

```
http://localhost:3000
```

Expected:

- redirected to `/login`

Case 2: Login with Google

Click:

```
Continue with Google
```

Expected:

- Google login opens
- after successful login → `/profile`

Case 3: Login with GitHub

Click:

```
Continue with GitHub
```

Expected:

- GitHub login opens

- after successful login → `/profile`

Case 4: Open `/profile` without login

Open:

```
http://localhost:3000/profile
```

Expected:

- redirected to `/login`

Case 5: Already logged in, open `/login`

Open:

```
http://localhost:3000/login
```

Expected:

- redirected to `/profile`

---

21) Most common mistakes (very important)

❌ Mistake 1: Installing wrong NextAuth version

Wrong:

```
npm install next-auth
```

Correct for this guide:

```
npm install next-auth@beta
```

Because this guide uses:

```
export const { handlers, auth, signIn, signOut } = NextAuth
(...)
```

That is **v5 style**.

❌ Mistake 2: Using `middleware.js` in Next.js 16

Wrong:

```
src/middleware.js
```

Correct:

```
src/proxy.js
```

❌ Mistake 3: Wrong proxy export

Wrong:

```
export { auth as default } from "@/auth";
```

Correct:

```
export const proxy = auth((req) => { ... });
```

Because Next.js 16 expects **a function**.

❌ Mistake 4: Wrong callback URLs

Google must be exactly:

```
http://localhost:3000/api/auth/callback/google
```

GitHub must be exactly:

```
http://localhost:3000/api/auth/callback/github
```

One wrong slash = login failure.

❌ Mistake 5: Wrong `.env.local` location

Correct:

```
project-root/.env.local
```

Wrong:

```
src/.env.local
```

❌ Mistake 6: Forgot restart after changing env

Always do:

```
rm -rf .next
npm run dev
```

---

22) What is happening internally?

Login flow

When user clicks:

```
signIn("google")
```

or

```
signIn("github")
```

NextAuth redirects user to provider login page.

After successful login:

- provider sends user back to:
  - `/api/auth/callback/google`
  - or `/api/auth/callback/github`

Then:

- NextAuth creates session
- stores auth cookie
- user becomes logged in

Route protection flow

When user opens `/profile`:

- `proxy.js` runs first
- checks if user is logged in
- if not logged in → redirect `/login`

That's why direct access is blocked.

---

23) Your setup is now working with:

- Google login
- GitHub login
- Protected `/profile`
- Login redirect logic
- Next.js 16 compatible auth flow

Best next upgrade (recommended)

Once this works, the next real-world step is:

1. **store logged-in user in database**
2. **show provider name (Google/GitHub)**
3. **custom navbar**
4. **logout from header**
5. **dashboard**
6. **role-based access**